Question: Which nutritional components (protein, fat, sodium, fiber, carbohydrates, sugars) best predict a cereal's calorie count, and how accurately can we estimate calories from these ingredients alone?

```
matMath = ▸ Object {transpose: ƒ(M), matMul: ƒ(A, B), invert: ƒ(M), ols: ƒ(X, y)}
```

```
matMath = {
  // Transpose a matrix
  const transpose = M => M[0].map((_, i) => M.map(row => row[i]));

  // Matrix multiply A x B
  const matMul = (A, B) => {
    const rows = A.length, cols = B[0].length, inner = B.length;
    return Array.from({length: rows}, (_, i) =>
      Array.from({length: cols}, (_, j) =>
        d3.sum(Array.from({length: inner}, (_, k) => A[i][k] * B[k][j]))
      )
    );
  };

  // Invert an n×n matrix via Gauss-Jordan elimination
  const invert = M => {
    const n = M.length;
    const A = M.map((row, i) => [...row, ...Array.from({length: n}, (_, j) => i === j ? 1 : 0)]);
    for (let col = 0; col < n; col++) {
      let pivot = A[col][col];
      A[col] = A[col].map(v => v / pivot);
      for (let row = 0; row < n; row++) {
        if (row !== col) {
          const factor = A[row][col];
          A[row] = A[row].map((v, k) => v - factor * A[col][k]);
        }
      }
    }
    return A.map(row => row.slice(n));
  };

  // OLS: given X (with intercept col) and y, return {beta, predicted, r2}
  const ols = (X, y) => {
    const Xt = transpose(X);
    const XtX = matMul(Xt, X);
    const XtX_inv = invert(XtX);
    const Xty = matMul(Xt, y.map(v => [v]));
    const beta = matMul(XtX_inv, Xty).map(v => v[0]);
    const predicted = X.map(row => d3.sum(row.map((v, i) => v * beta[i])));
    const yMean = d3.mean(y);
    const sst = d3.sum(y.map(v => (v - yMean) ** 2));
    const sse = d3.sum(y.map((v, i) => (v - predicted[i]) ** 2));
    return { beta, predicted, r2: 1 - sse / sst };
  };

  return { transpose, matMul, invert, ols };
}
```

```
regression = ▸ Object {clean: Array(77), predictorKeys: Array(6), beta: Array(7), predicted: Array(77), r2: 0.9187344902967236, stats: Obj
```

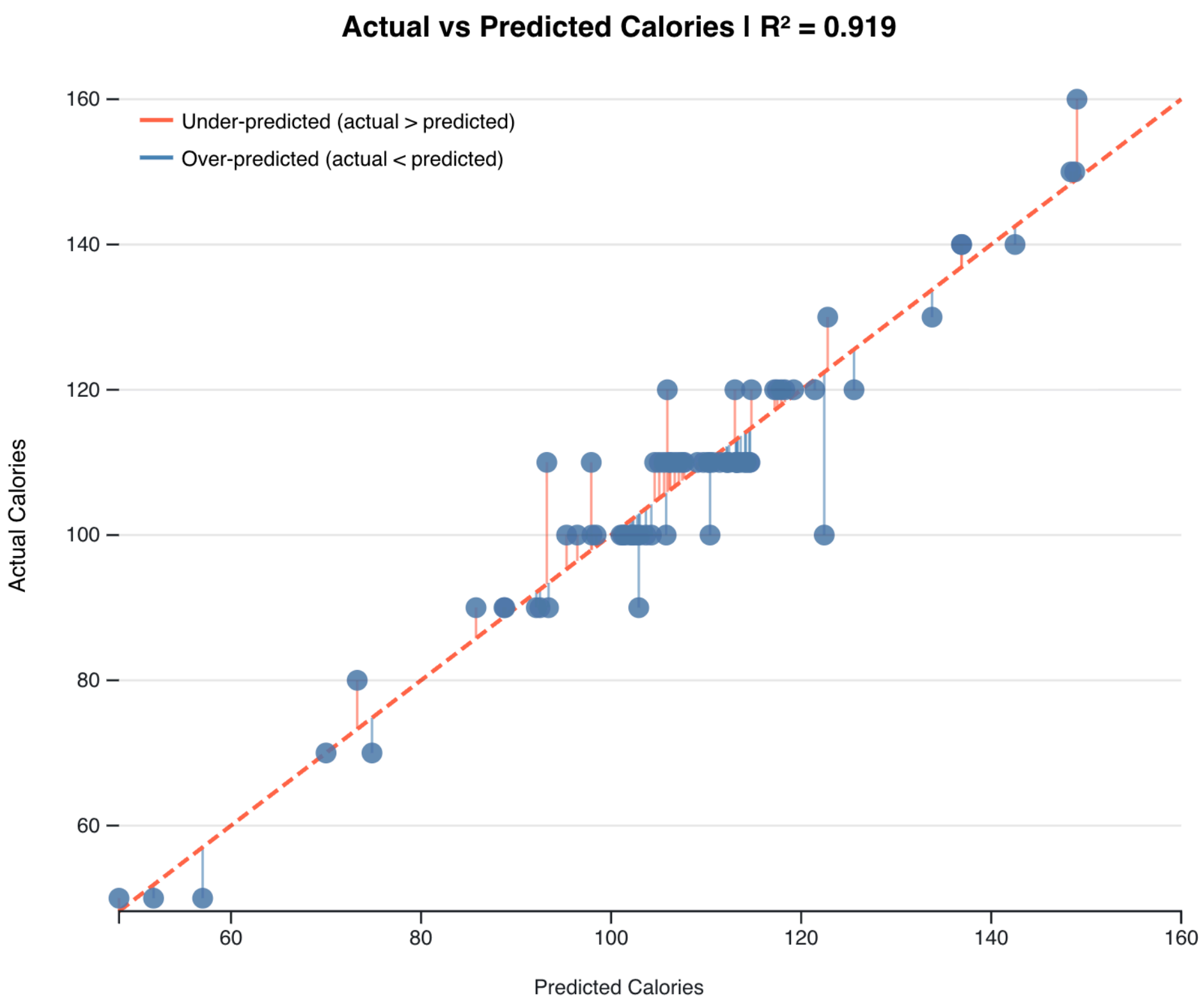
```
regression = {
  const predictorKeys = ["protein", "fat", "sodium", "fiber", "carbo", "sugars"];
  const clean = data.filter(d =>
    d.calories > 0 && d.sugars >= 0 && d.carbo >= 0 && d.protein >= 0
  );

  // Normalize each predictor: z = (x - mean) / std
  const stats = Object.fromEntries(predictorKeys.map(k => {
    const vals = clean.map(d => +d[k]);
    const mean = d3.mean(vals);
    const std  = d3.deviation(vals);
    return [k, { mean, std }];
  }));

  const normalize = (k, v) => (v - stats[k].mean) / stats[k].std;

  // X matrix with intercept col + normalized predictors
  const X = clean.map(d => [1, ...predictorKeys.map(k => normalize(k, +d[k]))]);
  const y = clean.map(d => +d.calories);

  const { beta, predicted, r2 } = matMath.ols(X, y);

  return {
    clean,
    predictorKeys,
    beta,
    predicted,
    r2,
    stats,
    labels: ["Intercept", ...predictorKeys]
  };
}
```

Regression Coefficients — Predicting Calories (Normalized Predictors)

Predictor	Std. Coef (β)	Mean	Std Dev	Raw β (per 1 unit)	Interpretation
Intercept	106.883	—	—	–	Predicted calories when all predictors are at their mean
protein	3.289	2.55	1.09	3.004	+3.29 cal per 1 SD increase in protein (= 3.00 cal per 1 unit)
fat	8.634	1.01	1.01	8.579	+8.63 cal per 1 SD increase in fat (= 8.58 cal per 1 unit)
sodium	0.621	159.68	83.83	0.007	+0.62 cal per 1 SD increase in sodium (= 0.01 cal per 1 unit)
fiber	0.863	2.15	2.38	0.362	+0.86 cal per 1 SD increase in fiber (= 0.36 cal per 1 unit)
carbo	15.750	14.81	3.88	4.058	+15.75 cal per 1 SD increase in carbo (= 4.06 cal per 1 unit)
sugars	16.868	7.03	4.35	3.878	+16.87 cal per 1 SD increase in sugars (= 3.88 cal per 1 unit)
R^2 = 0.9187		**Model explains 91.9% of calorie variance**			

Raw β: back-transformed to original units (e.g. cal per 1g fat, cal per 1mg sodium).

```
html`
<h3>Regression Coefficients — Predicting Calories (Normalized Predictors)</h3>
<table style="border-collapse:collapse; font-family:sans-serif; font-size:13px;">
  <thead>
    <tr style="background:#4C78A8; color:white;">
      <th style="padding:8px 16px; text-align:left;">Predictor</th>
      <th style="padding:8px 16px; text-align:right;">Std. Coef (β)</th>
      <th style="padding:8px 16px; text-align:right;">Mean</th>
      <th style="padding:8px 16px; text-align:right;">Std Dev</th>
      <th style="padding:8px 16px; text-align:right;">Raw β (per 1 unit)</th>
      <th style="padding:8px 16px; text-align:left;">Interpretation</th>
    </tr>
  </thead>
  <tbody>
    ${regression.labels.map((label, i) => {
      const isIntercept = i === 0;
      const key = regression.predictorKeys[i - 1];
      const s = isIntercept ? null : regression.stats[key];
      const rawBeta = isIntercept ? null : regression.beta[i] / s.std;
      return `
        <tr style="background:${i % 2 ? '#f5f8ff' : 'white'}">
          <td style="padding:6px 16px; font-weight:bold;">${label}</td>
          <td style="padding:6px 16px; text-align:right; font-family:monospace;">
            ${regression.beta[i].toFixed(3)}
          </td>
          <td style="padding:6px 16px; text-align:right; color:#888;">
            ${isIntercept ? "—" : s.mean.toFixed(2)}
          </td>
          <td style="padding:6px 16px; text-align:right; color:#888;">
            ${isIntercept ? "—" : s.std.toFixed(2)}
          </td>
          <td style="padding:6px 16px; text-align:right; font-family:monospace; color:#555;">
            ${isIntercept ? "—" : rawBeta.toFixed(3)}
          </td>
          <td style="padding:6px 16px; color:#555;">
            ${isIntercept
              ? "Predicted calories when all predictors are at their mean"
              : `+${regression.beta[i].toFixed(2)} cal per 1 SD increase in ${label} (= ${rawBeta.toFixed(2)} cal per 1 unit)`}
          </td>
        </tr>
      `;
    }).join("")}
    <tr style="background:#e8f0fe; font-weight:bold;">
      <td style="padding:8px 16px;" colspan="2">R² = ${regression.r2.toFixed(4)}</td>
      <td style="padding:8px 16px; color:#333;" colspan="4">
        Model explains ${(regression.r2 * 100).toFixed(1)}% of calorie variance
      </td>
    </tr>
  </tbody>
</table>
<p style="font-size:11px; color:#888; font-family:sans-serif;">
  Std. Coef (β): effect of a 1 standard deviation increase — comparable across predictors.<br>
  Raw β: back-transformed to original units (e.g. cal per 1g fat, cal per 1mg sodium).
</p>`
```


Actual vs Predicted Calories | R² = 0.919
Under-predicted (actual > predicted)
Over-predicted (actual < predicted)
Actual Calories
Predicted Calories
160
140
120
100
80
60
60
80
100
120
140
160

```
chart_regression = {
  const { clean, predicted, r2 } = regression;
  const width = 600, height = 500;
  const margin = {top: 50, right: 30, bottom: 60, left: 60};

  const allVals = [...clean.map(d => d.calories), ...predicted];
  const extent = d3.extent(allVals);

  const x = d3.scaleLinear(extent, [margin.left, width - margin.right]);
  const y = d3.scaleLinear(extent, [height - margin.bottom, margin.top]);

  const svg = d3.create("svg")
    .attr("viewBox", [0, 0, width, height])
    .attr("style", "max-width:100%; height:auto; font:11px sans-serif;");

  // ── 1. Residual lines: blue dot → red line (y=x means landing at predicted,predicted)
  svg.append("g")
    .selectAll("line")
    .data(clean)
    .join("line")
    .attr("x1", (d, i) => x(predicted[i]))   // start at point x
    .attr("y1", (d, i) => y(d.calories))      // start at actual y
    .attr("x2", (d, i) => x(predicted[i]))   // same x (vertical drop)
    .attr("y2", (d, i) => y(predicted[i]))   // land on red line where y = predicted
    .attr("stroke", d => d.calories > predicted[clean.indexOf(d)] ? "tomato" : "steelblue")
    .attr("stroke-width", 1.2)
    .attr("stroke-opacity", 0.6);

  // ── 2. Perfect prediction line (y = x)
  svg.append("line")
    .attr("x1", x(extent[0])).attr("y1", y(extent[0]))
    .attr("x2", x(extent[1])).attr("y2", y(extent[1]))
    .attr("stroke", "tomato").attr("stroke-width", 2)
    .attr("stroke-dasharray", "6,3");

  // ── 3. Blue dots (rendered on top of lines)
  svg.append("g")
    .selectAll("circle")
    .data(clean)
    .join("circle")
    .attr("cx", (d, i) => x(predicted[i]))
    .attr("cy", d => y(d.calories))
    .attr("r", 5)
    .attr("fill", "#4C78A8")
    .attr("opacity", 0.85)
    .append("title")
    .text((d, i) => `${d.name}\nActual: ${d.calories} | Predicted: ${predicted[i].toFixed(1)}\nResidual: ${(d.calories -
predicted[i]).toFixed(1)}`);
```

```
  // ── 4. Axes
  svg.append("g")
    .attr("transform", `translate(0,${height - margin.bottom})`)
    .call(d3.axisBottom(x).ticks(6))
    .call(g => g.append("text")
      .attr("x", width / 2).attr("y", 40)
      .attr("fill", "currentColor").attr("text-anchor", "middle")
      .text("Predicted Calories"));
  svg.append("g")
    .attr("transform", `translate(${margin.left},0)`)
    .call(d3.axisLeft(y).ticks(6))
    .call(g => g.select(".domain").remove())
    .call(g => g.selectAll(".tick line")
      .clone().attr("x2", width - margin.left - margin.right)
      .attr("stroke-opacity", 0.1));
  svg.append("text")
    .attr("transform", "rotate(-90)")
    .attr("x", -height / 2).attr("y", 15)
    .attr("text-anchor", "middle").text("Actual Calories");

  // ── 5. Title
  svg.append("text")
    .attr("x", width / 2).attr("y", 20)
    .attr("text-anchor", "middle").attr("font-weight", "bold").attr("font-size", "14px")
    .text(`Actual vs Predicted Calories  |  R² = ${r2.toFixed(3)}`);

  // ── 6. Legend
  const lg = svg.append("g").attr("transform", `translate(${margin.left + 10}, ${margin.top + 10})`);
  [["tomato", "Under-predicted (actual > predicted)"], ["steelblue", "Over-predicted (actual < predicted)"]].forEach(([c, label], i) => {
    lg.append("line").attr("x1", 0).attr("y1", i * 18).attr("x2", 16).attr("y2", i * 18)
      .attr("stroke", c).attr("stroke-width", 2);
    lg.append("text").attr("x", 20).attr("y", i * 18 + 4).text(label).attr("font-size", "10px");
  });

  return svg.node();
}
```

```
chart_small_multiples = ▸ Array(6) [SVGSVGElement, SVGSVGElement, SVGSVGElement, SVGSVGElement, SVGSVGElement, SVGSVGElement]
```

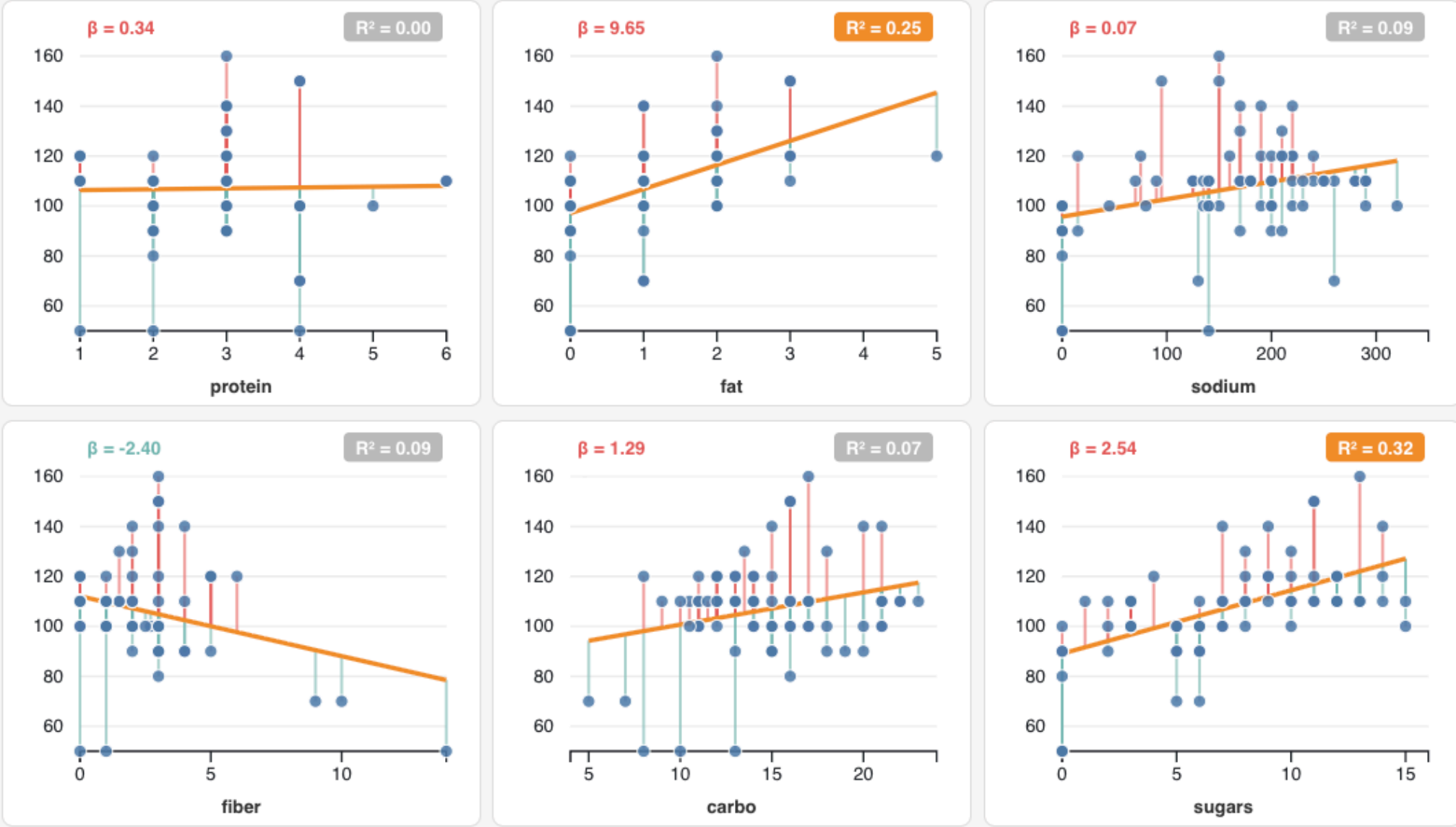
```
chart_small_multiples = {
  const raw = await FileAttachment("cereal.csv").csv({typed: true});
  const clean = raw.filter(d => +d.calories > 0 && +d.sugars >= 0 && +d.carbo >= 0);
  const keys = ["protein", "fat", "sodium", "fiber", "carbo", "sugars"];

  function ols(xs, ys) {
    const mx = d3.mean(xs), my = d3.mean(ys);
    let num = 0, den = 0;
    for (let i = 0; i < xs.length; i++) {
      num += (xs[i] - mx) * (ys[i] - my);
      den += (xs[i] - mx) ** 2;
    }
    const slope = num / den;
    const intercept = my - slope * mx;
    const yhat = xs.map(xi => intercept + slope * xi);
    const sst = d3.sum(ys.map(v => (v - my) ** 2));
    const sse = d3.sum(yhat.map((yh, i) => (ys[i] - yh) ** 2));
    return { slope, intercept, r2: 1 - sse / sst, yhat };
  }

  const W = 260, H = 220, mt = 30, mr = 18, mb = 40, ml = 42;

  return keys.map(key => {
    const xs = clean.map(d => parseFloat(d[key]));
    const ys = clean.map(d => parseFloat(d.calories));
    const { slope, intercept, r2, yhat } = ols(xs, ys);
    const xExt = d3.extent(xs);
    const yExt = d3.extent(ys);
    const x = d3.scaleLinear().domain(xExt).nice().range([ml, W - mr]);
    const y = d3.scaleLinear().domain(yExt).nice().range([H - mb, mt]);

    const svg = d3.create("svg")
      .attr("width", W).attr("height", H)
      .attr("style", "font:10px sans-serif; background:white; border:1px solid #ddd; border-radius:6px; display:block;");

    // Y axis
    const yg = svg.append("g").attr("transform", `translate(${ml},0)`);
    yg.call(d3.axisLeft(y).ticks(4));
    yg.select(".domain").remove();
    yg.selectAll(".tick line").attr("x2", W - ml - mr).attr("stroke", "#eee");

    // X axis
    svg.append("g").attr("transform", `translate(0,${H - mb})`).call(d3.axisBottom(x).ticks(4));

    // Residual lines
    for (let i = 0; i < clean.length; i++) {
      const cx = x(xs[i]), yTop = y(ys[i]), yBot = y(yhat[i]);
      if (isNaN(cx) || isNaN(yTop) || isNaN(yBot) || Math.abs(yTop - yBot) < 1) continue;
      svg.append("path")
        .attr("d", `M ${cx} ${yTop} L ${cx} ${yBot}`)
        .attr("stroke", ys[i] > yhat[i] ? "#e45756" : "#72b7b2")
        .attr("stroke-width", 1.5).attr("fill", "none").attr("opacity", 0.6);
    }
```

```
    // Fit line
    svg.append("path")
      .attr("d", `M ${x(xExt[0])} ${y(intercept + slope * xExt[0])} L ${x(xExt[1])} ${y(intercept + slope * xExt[1])}`)
      .attr("stroke", "#f28e2b").attr("stroke-width", 2.5).attr("fill", "none");

    // Dots
    for (let i = 0; i < clean.length; i++) {
      const cx = x(xs[i]), cy = y(ys[i]);
      if (isNaN(cx) || isNaN(cy)) continue;
      svg.append("circle")
        .attr("cx", cx).attr("cy", cy).attr("r", 3.5)
        .attr("fill", "#4C78A8").attr("stroke", "white").attr("stroke-width", 0.8).attr("opacity", 0.85)
        .append("title")
        .text(`${clean[i].name}\n${key}: ${xs[i]}, cal: ${ys[i]}\nresidual: ${(ys[i] - yhat[i]).toFixed(1)}`);
    }

    // Predictor label
    svg.append("text").attr("x", W / 2).attr("y", H - 6)
      .attr("text-anchor", "middle").attr("font-weight", "bold").attr("fill", "#333").text(key);

    // R² badge
    const bc = r2 > 0.5 ? "#4C78A8" : r2 > 0.2 ? "#f28e2b" : "#bbb";
    svg.append("rect").attr("x", W - mr - 56).attr("y", 6).attr("width", 54).attr("height", 16).attr("rx", 3).attr("fill", bc);
    svg.append("text").attr("x", W - mr - 29).attr("y", 18)
      .attr("text-anchor", "middle").attr("fill", "white").attr("font-size", "10px").attr("font-weight", "bold")
      .text(`R² = ${r2.toFixed(2)}`);

    // β label
    svg.append("text").attr("x", ml + 4).attr("y", 18)
      .attr("fill", slope >= 0 ? "#e45756" : "#72b7b2").attr("font-size", "10px").attr("font-weight", "bold")
      .text(`β = ${slope.toFixed(2)}`);

    return svg.node();
  });
}
```

html`

Small Multiples — Each Predictor vs Calories

Each panel shows a simple linear regression of one nutritional predictor against calories. Colored vertical lines are residuals. Orange line is the fit. R^2 badge indicates model strength.



`